K. J. Somaiya College of Engineering, Mumbai-77

Batch: C-3	**Roll No.: 16010123325**
Experiment No. 1	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

Title: Introduction to Matlab

Objective: To familiarize the beginner to MATLAB by introducing the basic features and commands of the program.

Expected Outcome of Experiment:

CO	Outcome
CO1	Identify various discrete time signals and systems and perform signal manipulation

Books/ Journals/ Websites referred:

1. http://www.mathworks.com/support/
2. www.math.mtu.edu/~msgocken/intro/intro.html
3. www.mccormick.northwestern.edu/docs/efirst/matlab.pdf

Pre Lab/ Prior Concepts:

INTRODUCTION TO MATLAB

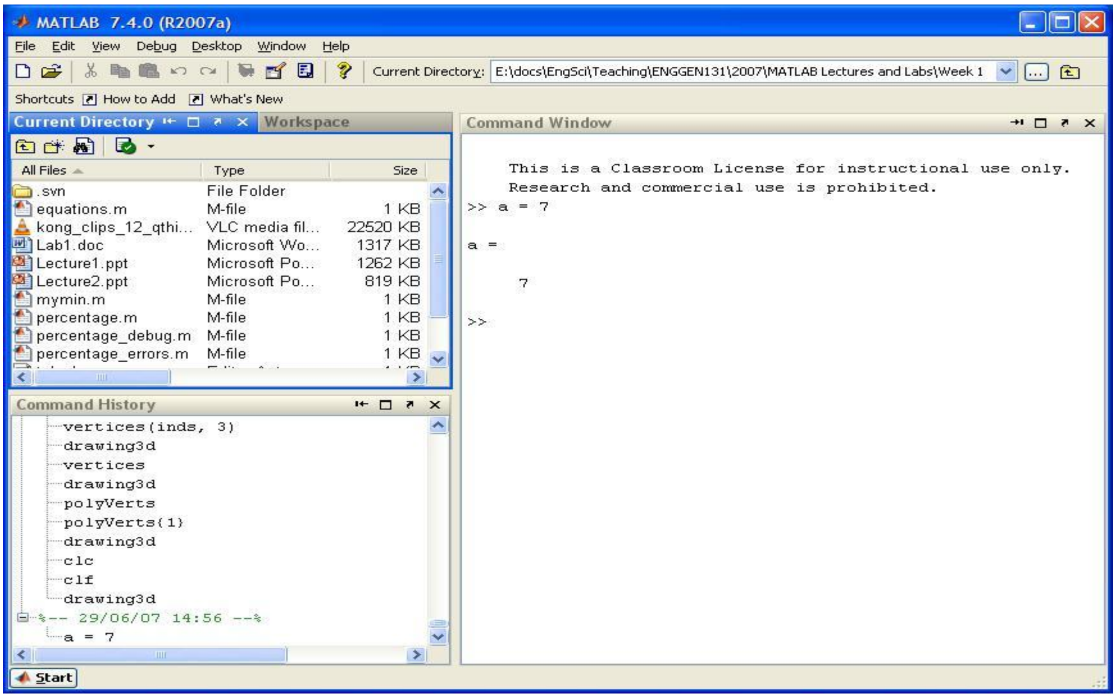
MATLAB (MATrix LABoratory) is an interactive software system for numerical computations and graphics. As the name suggests, MATLAB is especially designed for matrix computations: solving systems of linear equations, performing matrix transformations, factoring matrices, and so forth. In addition, it has a variety of graphical capabilities, and can be extended through programs written in its own programming language.

KEY FEATURES

- High-level language for numerical computation, visualization, and application development.
- Interactive environment for iterative exploration, design, and problem solving.
- Mathematical functions for linear algebra, statistics, Fourier analysis, filtering, optimization, numerical integration, and solving ordinary differential equations.
- Built-in graphics for visualizing data and tools for creating custom plots.
- Development tools for improving code quality and maintainability and maximizing performance.
- Tools for building applications with custom graphical interfaces.
- Functions for integrating MATLAB based algorithms with external applications and languages such as C, Java, .NET.

GETTING STARTED

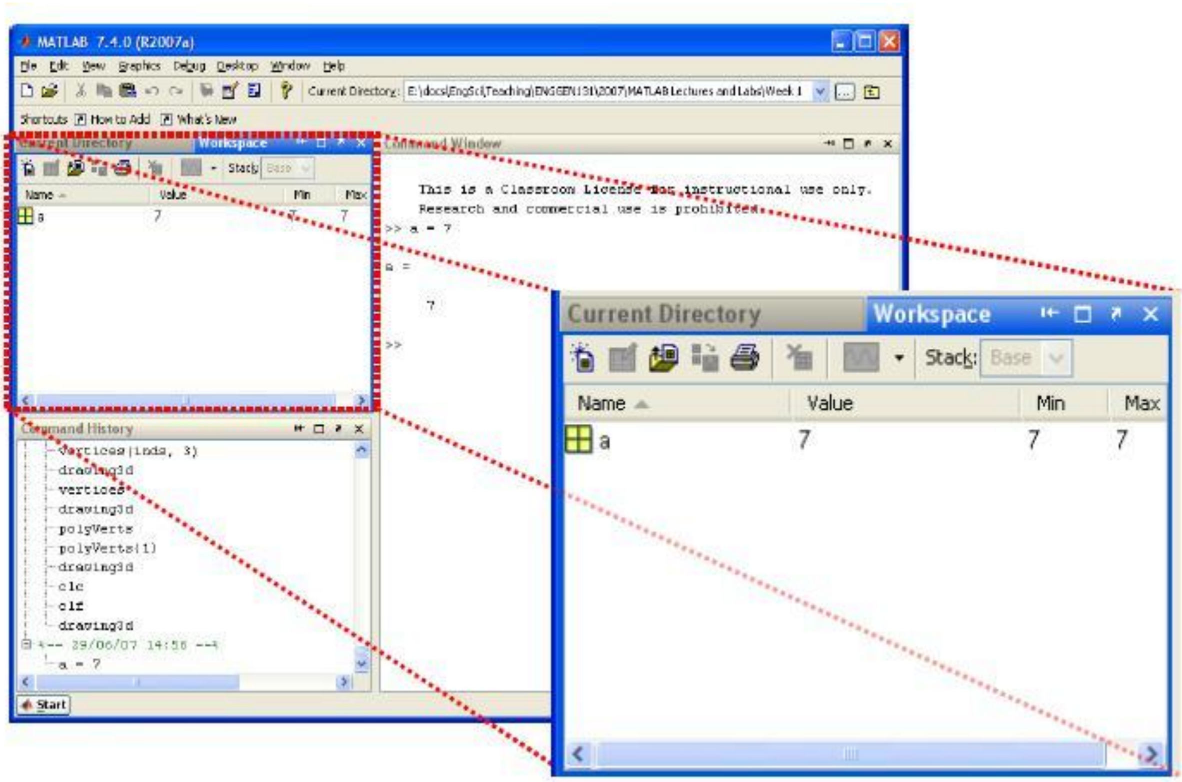
Double click on the MATLAB icon. The MATLAB window should come up on your screen. It looks like this:



The main window on the right is called the *Command* Window. This is the window in which you interact with MATLAB. Once the MATLAB prompt >> is displayed, all MATLAB commands are executed from this window. In the figure, you can see that we execute the command:

>> a = 7

In the top left corner you can view the *Workspace* window and the *Current Directory* window. Swap from one to the other by clicking on the appropriate tab. Note that when you first start up MATLAB, the workspace window is empty. However, as you execute commands in the *Command* window, the *Workspace* window will show all variables that you create in your current MATLAB session. In this example, the workspace contains the variable **a**.



In the bottom left corner you can see the *Command History* window, which simply gives a chronological list of all MATLAB commands that you used.

During the MATLAB sessions you will create files to store programs or workspaces. Therefore create an appropriate folder to store the lab files.

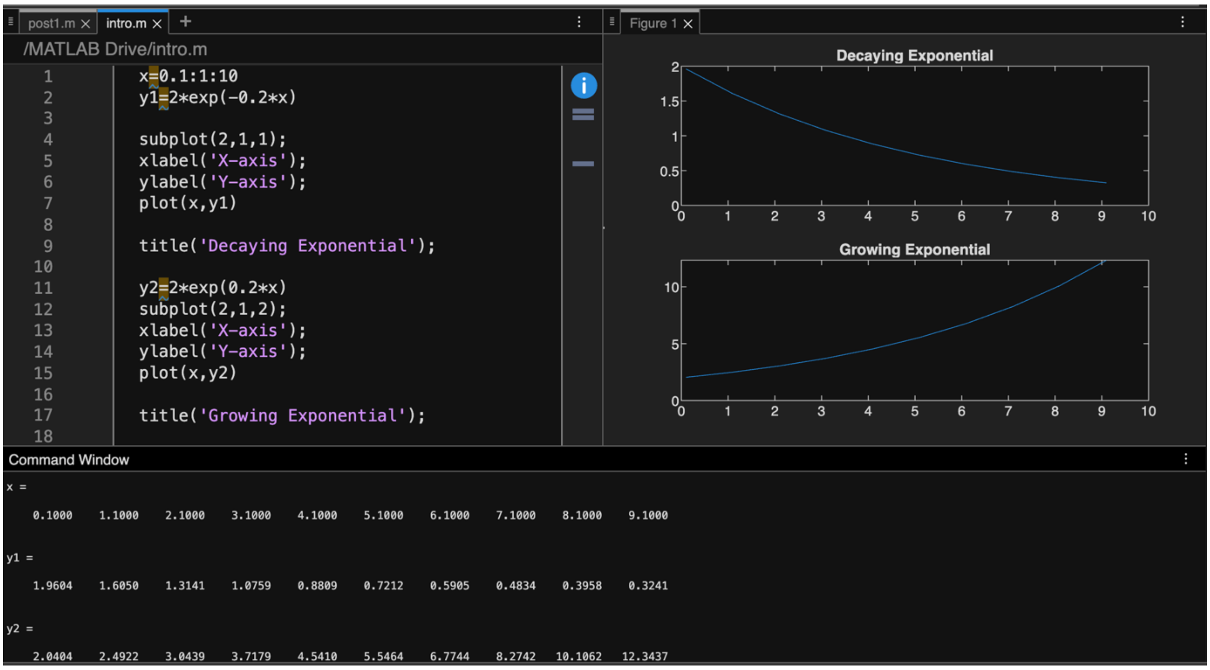
THE MATLAB HELP SYSTEM

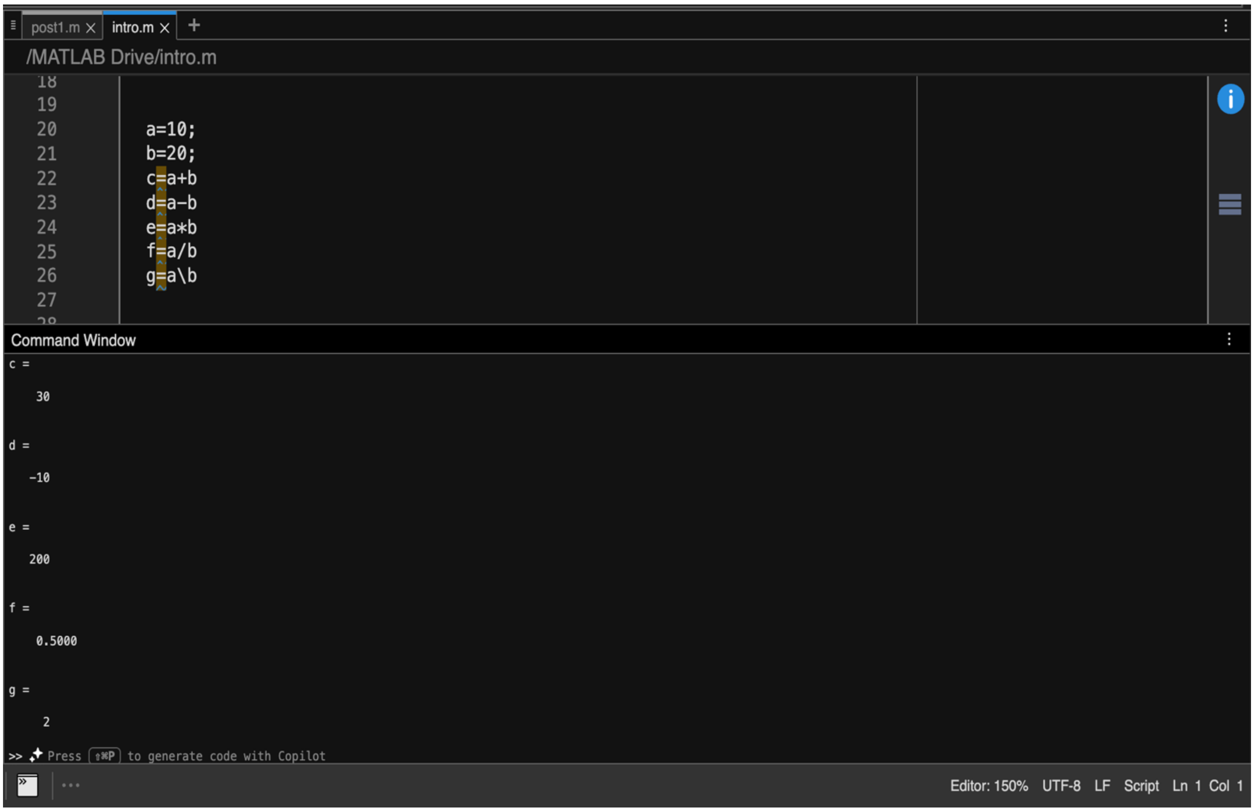
MATLAB's help system provides information to assist you while using MATLAB. Select *Help for MATLAB Help* to open the help system.

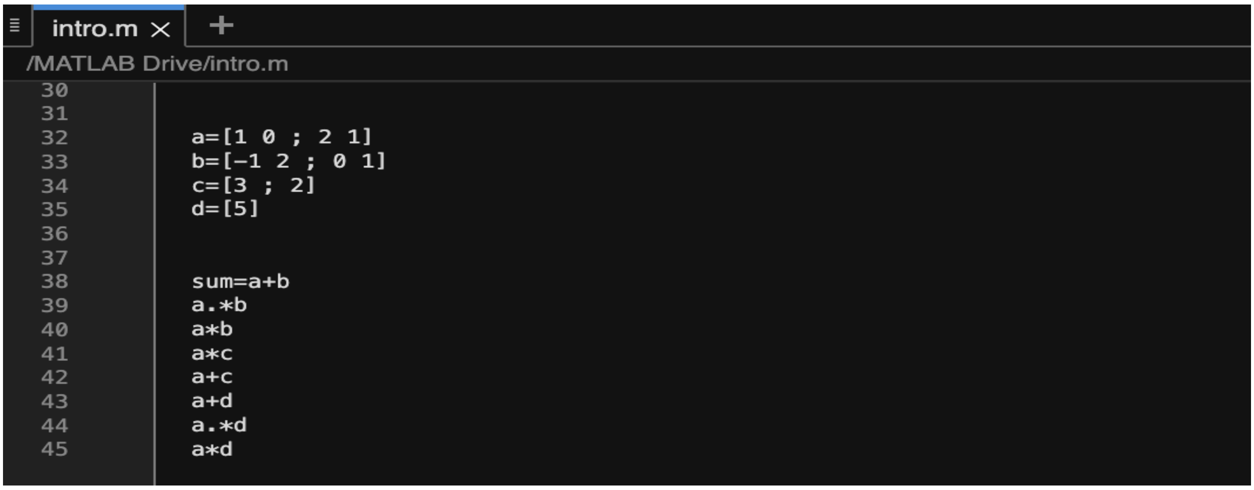
We can then browse the commands via the *Contents* window, look through the *Index* of commands or *Search* for keywords or phrases within the documentation. This is the most comprehensive documentation for MATLAB and is the best place to find out what you need. You can also start the MATLAB *Help* using the **helpwin** command.

A faster way to find out information about commands is via the **help** command. If you just type **help** and hit return, you will be presented with a menu of help options. If you type **help <command>**, MATLAB will provide help for that particular command. For example, **help hist** will call up information on the **hist** command, which produces histograms. Note that the **help** command only displays a short summary of how to use the command you are interested in. For more detailed help documentation, use the **doc** command. For example, **doc hist** will display the full documentation of the **hist** command, including examples of how to use it and sample output.

Matlab Commands used for Signal and Image processing:




post1.m
intro.m
/MATLAB Drive/intro.m
a=10;
b=20;
c=a+b
d=a-b
e=a*b
f=a/b
g=a\b
Command Window
c =
30
d =
-10
e =
200
f =
0.5000
g =
2
>> Press to generate code with Copilot
Editor: 150% UTF-8 LF Script Ln 1 Col 1


intro.m
/MATLAB Drive/intro.m
a=[1 0 ; 2 1]
b=[-1 2 ; 0 1]
c=[3 ; 2]
d=[5]
sum=a+b
a.*b
a*b
a*c
a+c
a+d
a.*d
a*d

```
post1.m   intro.m
Command Window

sum =

     0     2
     2     2


ans =

    -1     0
     0     1


ans =

    -1     2
    -2     5


ans =

     3
     8


ans =

     4     3
     4     3


ans =

     6     5
     7     6


ans =

     5     0
    10     5
```

```
ans =

     5     0
    10     5

>>
```

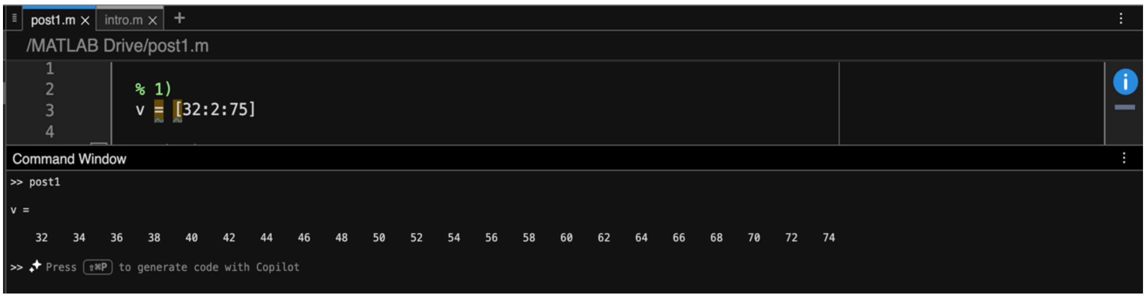
Conclusion:-

This experiment highlights the basic operations that can be performed on MATLAB, including matrix operations and gives an introduction to MATLAB.

Date: _____________ **Signature of faculty in-charge**

Post Lab Questions

1. Create a vector of the even whole numbers between **31 and 75.**



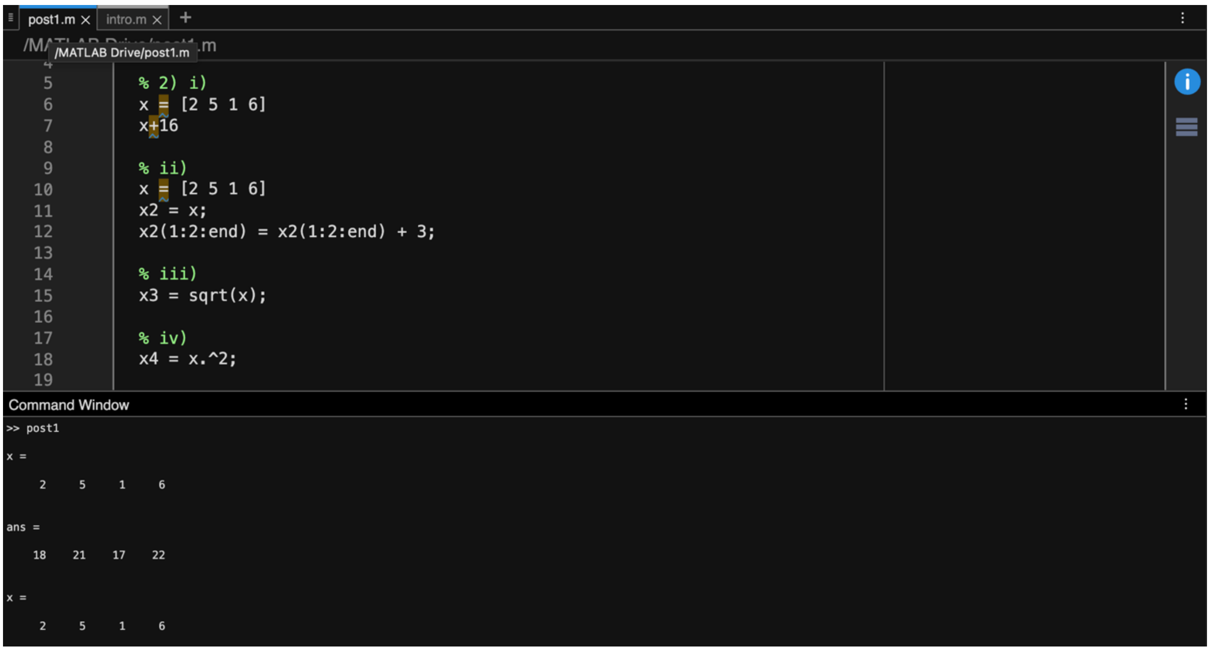
2. Let x = **[2 5 1 6].**

   a. Add 16 to each element

   b. Add 3 to just the odd-index elements

   c. Compute the square root of each element

   d. Compute the square of each element



3. Let x = **[3 2 6 8]'** and y = **[4 1 3 5]'** (NB. x and y should be column vectors).

   a. Add the sum of the elements in x to y

   b. Raise each element of x to the power specified by the corresponding element in y.

   c. Divide each element of y by the corresponding element in x

   d. Multiply each element in x by the corresponding element in y, calling the result "z".

   e. Add up the elements in z and assign the result to a variable called "w".

   f. Compute x' *y - w and interpret the result

post1.m intro.m

/MATLAB Drive/post1.m

```
% 3)
x = [3; 2; 6; 8];
y = [4; 1; 3; 5];

% a)
y1 = y + sum(x)

% b)
p = x.^y

% c)
d = y ./ x

% d)
z = x .* y

% e)
w = sum(z)

% f)
result = x' * y - w
```

```
y1 =

    23
    20
    22
    24


p =

          81
           2
         216
       32768


d =

    1.3333
    0.5000
    0.5000
    0.6250


z =

    12
     2
    18
    40


w =

    72


result =

     0
```

4 Create a vector x with the elements, $x_n = (-1)^{n+1}/(2n-1)$

Add up the elements of the version of this vector that has 100 elements.

```
post1.m × intro.m × +
/MATLAB Drive/post1.m
% % 4)
n = 1:100;
x = (-1).^(n+1) ./ (2*n - 1);
S = sum(x)
%
Command Window
>> post1

S =

   0.7829
```

5. Given the array **A = [2 4 1; 6 7 2; 3 5 9],** provide the commands needed to

   a. Assign the first row of A to a vector called x1

   b. Assign the last 2 rows of A to an array called y

   c. Compute the sum over the columns of A

   d. Compute the sum over the rows of A

   e. Compute the standard error of the mean of each column of A (NB. the standard error of the mean is defined as the standard deviation divided by the square root of the number of elements used to compute the mean.)

post1.m intro.m

/MATLAB Drive/post1.m

```
% 5)
A = [2 4 1;
     6 7 2;
    3 5 9];

% i)
x1 = A(1, :)

% ii)
y = A(2:3, :)

% iii)
col_sum = sum(A)

% iv)
row_sum = sum(A, 2)

% v)
sem = std(A) ./ sqrt(size(A,1))
```

Command Window

```
>> post1

x1 =

     2     4     1


y =

     6     7     2
     3     5     9


col_sum =

    11    16    12


row_sum =

     7
    15
    17


sem =

    1.2019    0.8819    2.5166
```